Lab No: 09

❋ **Lab Title:** **Functions Part I**

○ **Learning objectives:**

The following topics will be covered in this lab session

- Function Introduction
- Function Syntax

↳ Functions:

Functions let you chop up a long program into named sections so that the sections can be reused throughout the program. Function has a name identifier, accepts **parameters** and **returns** a result. C++ functions can accept an unlimited number of parameters. In general, C++ does not care in what order you put your functions in the program, so long as the function name is known to the compiler before it is called.

We have been consistently using library functions in the programs such as getch() and clrscr(). We just simply use these functions without knowing what's inside these functions. We just call them from our program with some parameters inside the brackets and it does the work for us. We don't know its implementation. When we actually call these functions, the code inside these functions executes and at the end it transfers the control back to the program from where the function was called.

In this lab we would learn how to make our own functions which can perform certain tasks. Function is differentiated from a simple variable, by the parenthesis just after the name as **main()**. It just indicates to the compiler that it's a function.

The most common function that you have encountered is **main()** which you have been implementing throughout the lab. **main()** is the function whose name identifier is **main** with no parameters passed to it. Anything inside brackets are the parameters passed to the function. In case of multiple parameters, each of them are separated with a comma(,) in between. Parameters are written as the comma separated list within brackets just after the name of function. In **main()** function there are no parameters passed to it as the brackets after the name is empty. As you know when we use **int main()** in our program then we return an integer from the main function. Similarly when we define our own functions in the program, that function should return appropriate value according to the data type which is given before the name of function when we give its definition or implementation, just as we return an integer when we use **int**

main() and nothing in case of **void main()**. C++ assumes that every function will return a value. If the programmer wants a return value, this is achieved using the return statement. If no return value is required, none should be used when calling the function.

- Function is written as:

Return_typefunction_name (comma separated list of parameters);

For example:

i. int main();

ii. float divide(int a, int b);

iii. void square(float x);

Here names of the functions are main, divide and square which take 0, 2 and 1 parameters respectively.

These functions would return an integer, float and void(nothing) respectively.

The **prototype** of a function is a way of describing the parameters (also called as arguments of function) and parameter types with which a legal call to the function can be made. It contains the name of the function, its parameters and their type, and the return value.

Pattern to write the function prototype is shown below.

Return_typefunction_name (comma separated list of parameters);

Three examples shown above are examples of the function prototype. If you are asked to write the prototype of any function it can be written as in the examples above.

If you need to do something again and again in the program after random time intervals (need to reuse same code again and again) you should use function that should implement that task what is required.

And whenever that task is needed to be performed that function is just needed to be called and task would be done. It would also decrease the size of the code and increases the reusability.

Lets do an example which calls a function which prints ten asterisks (*) in line. (**********)

In this example main function just calls the function named **asterisks** which just prints the line of ten asterisks.

In this example main function just calls the function named **asterisks** which just prints the line of ten asterisks.

```
//This program is a simple function sample
void asterisk();  //function prototype
void main()
{
        clrscr();
        asterisk();        //function call
        getch();
}
// function body
void asterisk()
{
        for (inti=0; i<10;i++)
        {
                cout<<"*";
        }
}
```

Second line in the example is the prototype declaration of the function because before calling the function it's must that you should at least provide function prototype as given in example. return_type function name(arguments separated by a comma);

Line 4 is the line which is calling the function.

asteriks() function in the previous example does not take any argument and does not return any thing represented by empty braces() after function name and void before function name respectively.

After main the function definition of the function asterisks is given.

Let's go one step ahead, function asterisks (int a) with a single argument.

```
//This program is a simple function sample
void asterisk(int n);  //function prototype
void main(){
        clrscr();
        asterisk(12);        //function call
        getch();
}// function body
void asterisk(intnum)
{
        for (inti=0; i<num;i++)
        {
                cout<<"*";
        }
```

This function would take an integer number as an argument and prints the no of asterisks equal to the number given as argument to the function in straight line. As in example we have given 7 as an argument while calling so it would print seven asterisks in line as 7 is passed to the function, intnum variable in function definition becomes equal to 7. As it is given in function prototype, compiler expects function definition should take single argument which should be of integer data type. Due to this in function calling you need to give an integer as argument (parameter) of a function.

Now an example of a function with two parameters.

```
//Function that adds two numbers
void add(int n1, int n2);  //function prototype ….this can also be written as
void add(int, int);
void main()
{
        clrscr();
        int answer;
        answer=add(3,5);        //function call
        cout<<answer;
        getch();
}

// function body
int add(int num1, int hum2)
{
        Return num1+num2;     // int sum; sum=num1+num2; return sum;
}
```

The above example can also be written as follow

```
//Function that adds two numbers
int add(int num1, int hum2)
{
        Return num1+num2;     // int sum; sum=num1+num2; return sum;
}
void main()
{
        clrscr();
        int answer;
        answer=add(3,5);        //function call
        cout<<answer;
        getch();
}
```

The above function calculates the sum of two numbers passed as arguments to the function and returns the sum to the calling function.

This function would be called from the main() function as

s = add(a , b);

where s, a, and b all would be integers declared in the main() function. When control in the main function would reach to *add(a,b)* then, add function would be called and control would be transferred to the code of add function and num1 and num2 of the ***add*** function would become equal to a and b respectively.

The values of *num1,*and *num2* are passed by the *main* function. The variable *sum* which is declared inside the *add* function stores the sum of the numbers and *return* statement is used to return the result calculated. You can declare any variable inside the function definition as it is done in main() as it is shown in the example above.

int add (int num1,int num2)

This line begins the function definition. It tells us the type of the return value, the name of the function, and a list of arguments used by the function. The arguments and their types are enclosed in brackets, each pair separated by commas.

The body of the function is bounded by a set of curly brackets. Any variables declared here will be treated as local unless specifically declared as static or extern types.

Return sum;

On reaching a return statement, the control of the program returns to the calling function. Value of sum is returned from the add function to the main (calling function) at.

s = add(a , b);

it would become s = (value returned from the add function which is sum);

If the final closing curly bracket in the add function's definition is reached before any return value, then the function will return automatically, any return value will then be meaningless.

Note: You cannot define the function inside another function, can only call another function from inside another function.

intis_even(int n); ------prototype

can also be witten as *intis_even (int);*

Lab Tasks

- **Task 1: Function Introduction and Declaration:**

1. Declare a function called "greet" that takes no parameters and returns nothing.
2. Inside the function, display a greeting message of your choice.
3. Call the function from the main program to execute the greeting.

Input:

```
include <iostream>   // Input/output ke liye library
using namespace std;  // std ko avoid karne ke liye

// Function declaration (prototype)
// Yeh batata hai ke greet naam ka function hai
void greet();

#
    int main()
{
   // Function call
   // Yahan greet function execute hoga
   greet();

 return 0;   // Program successfully end
  }

// Function definition
      void greet()
{
   // Message display kar raha hai
   cout << "Assalam-o-Alaikum! Welcome to C++ Programming!"
     << endl;
}
```

Output:

Assalam-o-Alaikum! Welcome to C++ Programming!

- **Task 2: Basic Function Declaration and Call:**

    1. Declare a function called "sayHello" that takes no parameters and returns nothing.
    2. Inside the function, display the message "Hello, World!".
    3. Call the function from the main program to execute the greeting.

Input:

```
#include <iostream>   // Input/output library
using namespace std;  // std ko avoid karne ke liye

// Function declaration
// Yeh function koi parameter nahi leta
void sayHello();

int main()
{
   // Function call
   // sayHello function yahan execute hoga
   sayHello();

   return 0;   // Program end
}

// Function definition
void sayHello()
\{
   // Simple message print
   cout << "Hello, World!" << endl;
}
```

Output:

```
Hello, World!
```

- **Task 3: Write a C++ program that:**

Takes 10 integer values as input from the user and stores them in a 1D array.

Creates a function named calculateSum that:

Accepts the array and its size as parameters Returns the sum of all elements In the main() function:

Call the function Calculate and display the average of the array elements Constraints: Use only one-dimensional array Use function for sum calculation

```
#include <iostream>   // Input/Output ke liye library
using namespace std;  // std likhne se bachne ke liye

// Function declaration (prototype)
// Yeh batata hai ke function ka naam kya hai aur kya input/output hoga
int calculateSum(int arr[], int size);

int main()
{
   const int SIZE = 10;   // Array ka fixed size (change nahi ho sakta)

   int numbers[SIZE];     // 10 integers ka array banaya

   int sum;               // Sum store karne ke liye variable
   double average;        // Average store karne ke liye (decimal value)

   // User se 10 numbers lene ke liye loop
   cout << "Enter 10 integers:" << endl;

   for(int i = 0; i < SIZE; i++)   // Loop 0 se 9 tak chalega
   {
      cin >> numbers[i];           // Har input array ke index me store hoga
   }

   // Function call
   // Array aur uska size function ko pass kiya
   sum = calculateSum(numbers, SIZE);

   // Average calculate kiya
   // (double) lagaya taake decimal result aaye
   average = (double)sum / SIZE;

   // Output display
   cout << "Sum = " << sum << endl;
   cout << "Average = " << average << endl;

   return 0;   // Program successfully end
}

// Function definition
int calculateSum(int arr[], int size)
{
   int sum = 0;   // Sum ko 0 se start kiya

   // Loop array ke tamam elements ko add karega
   for(int i = 0; i < size; i++)
   {
      sum = sum + arr[i];   // Har element sum me add hota hai
   }

   return sum;   // Final sum wapas main function ko bhej diya
}
```

Output:

- **Sum = 55**

  Average = 5.5

Conclusion

In this lab, we learned how to create and use functions in C++. We practiced simple functions without parameters and also implemented a function with an array to calculate sum and average. This lab improved our understanding of modular programming and how functions make code efficient, readable, and reusable.